

PROPOSAL PROYEK AKHIR

MATA KULIAH KEAMANAN SISTEM INFORMASI

Implementasi dan Evaluasi Mitigasi Context-Aware Authorization Berbasis Zero Trust terhadap Broken Object Level Authorization (BOLA) pada REST API Multi-Tenant

Disusun oleh

Kelompok 5 :

Anggita Cristin Meylani	222312982
Kaylla Zahrani	222313161
Muhammad Imaddudin Zaki	222313244
Nadia Nur Nisrina	222313276
Nyimas Virna Salsa Lestari Risqia	222313307

Dosen Pengampu:

Farid Ridho, S.S.T., M.T.

PROGRAM STUDI DIV KOMPUTASI STATISTIK

POLITEKNIK STATISTIKA STIS

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Rumusan Masalah.....	4
1.3 Tujuan Proyek.....	5
1.5 Ruang Lingkup dan Batasan.....	6
BAB II TINJAUAN PUSTAKA.....	7
2.1 Landasan Teori.....	7
2.1.1. REST API dan Arsitektur Multi-Tenant.....	7
2.1.4. Otorisasi Berbasis Konteks (Context-Aware Authorization).....	10
2.1.5. Komponen Teknologi Implementasi.....	12
2.2 Penelitian/Proyek Terkait.....	13
BAB III METODOLOGI.....	15
3.1 Tahapan Pelaksanaan.....	15
3.1.1. Studi Literatur dan Analisis Kebutuhan.....	15
3.1.2. Perancangan Sistem dan Arsitektur Keamanan.....	17
3.1.3. Implementasi dan Pengembangan.....	20
3.1.4. Pengujian dan Evaluasi.....	22
3.1.5. Dokumentasi dan Penyusunan Laporan.....	22
3.2 Arsitektur/Desain Sistem.....	22
3.2.1. Diagram Arsitektur Sistem.....	22
3.2.2. Diagram Alir (Logika Context-Aware Authorization Engine).....	24
3.2.3. Justifikasi Bobot Risk Score dan Threshold.....	25
3.2.4. Struktur Kebijakan Otorisasi (OPA Rego).....	28
3.3 Skenario Pengujian.....	30
3.3.1. Lingkungan Pengujian.....	30
3.3.2. Skenario Pengujian.....	31
3.3.3. Parameter Keberhasilan.....	33
3.3.4. Evaluasi.....	34
BAB IV JADWAL PELAKSANAAN.....	36
BAB V PEMBAGIAN TUGAS.....	37
DAFTAR PUSTAKA.....	39

BAB I PENDAHULUAN

1.1 Latar Belakang

Kontrol akses masih menjadi tantangan utama dalam aspek keamanan sistem aplikasi modern. Laporan Open Web Application Security Project (OWASP) dalam OWASP Top 10: 2025 [1] menempatkan *Broken Access Control* (BAC) pada peringkat pertama (A01:2025) kerentanan aplikasi web dengan rata-rata tingkat kejadian (*Average Incidence Rate*) 3,74% dan total kemunculan *Common Weakness Enumeration* (CWE) yang terdokumentasi mencapai lebih dari 1.839.701 kasus. Tingginya angka penemuan ini menunjukkan bahwa mekanisme kontrol akses konvensional secara sistematis masih belum memadai.

Permasalahan ini menjadi lebih spesifik pada ekosistem *Application Programming Interface* (API) yang merupakan antarmuka dominan komunikasi antar layanan [2]. OWASP API Security Top 10:2023 [3] menempatkan *Broken Object Level Authorization* (BOLA) sebagai ancaman API paling kritis, yang telah menduduki posisi pertama sejak 2019. Eksploitasi BOLA terjadi ketika penyerang memanipulasi parameter *object ID* pada *request* API untuk mengakses sumber daya pengguna lain tanpa otorisasi. Kerentanan ini terjadi di tingkat objek, bukan di tingkat *endpoint*, sehingga mekanisme autentikasi yang sudah benar pun tidak cukup untuk mencegahnya [3]. Peneliti [4] yang menguji keamanan 25 aplikasi database-backed populer berhasil mengidentifikasi 193 kerentanan BOLA nyata, termasuk 178 kerentanan yang belum pernah dilaporkan sebelumnya. Hal ini menunjukkan bahwa kerentanan BOLA sangat tersebar luas bahkan pada aplikasi yang sudah matang dan aktif dipelihara.

Ancaman serangan BOLA meningkat secara signifikan dalam arsitektur multi-tenant, yakni ketika sejumlah entitas organisasi (*tenant*) berbagi infrastruktur basis data yang sama dan layanan komputasi yang sama (*shared schema*) [2]. Pada arsitektur ini, kerentanan BOLA tidak hanya mengancam satu pengguna, melainkan berpotensi memungkinkan pelanggaran isolasi lintas *tenant*, yaitu seorang pengguna dari *tenant* A dapat mengakses atau memanipulasi sumber daya milik *tenant* B hanya dengan mengubah *object ID* pada *request* REST API, tanpa memerlukan eskalasi hak akses apapun [3]. RESTful API yang terus-menerus terekspos di internet merupakan target umum serangan siber, dan BOLA secara langsung memungkinkan penyerang memodifikasi atau menghapus data serta menjalankan aksi yang seharusnya hanya boleh dilakukan oleh pengguna yang berwenang [1].

Meskipun berbagai mekanisme otorisasi telah diterapkan, celah struktural terhadap BOLA pada lingkungan *multi-tenant* masih signifikan. Model *Role-Based Access Control* (RBAC) yang dominan secara inheren hanya mengevaluasi peran, bukan kepemilikan objek atau atribut kontekstual. Hal ini membuatnya tidak memadai untuk mencegah *horizontal privilege escalation* lintas-tenant karena gagal memproses faktor dinamis seperti lokasi dan perilaku akses [5]. Di sisi lain, pendekatan analitik mutakhir juga memiliki batasan operasional. Deteksi BOLA melalui transformasi OpenAPI yang dikembangkan oleh [2] menunjukkan akurasi tinggi, namun beroperasi secara *post-hoc* dari *server log* sehingga tidak memberikan perlindungan *real-time*. Di samping itu, metode inferensi otorisasi seperti BolaRay dalam [4] rentan menghasilkan *false positive* karena model statisnya gagal beradaptasi dengan dinamika logika bisnis API. Hal ini menegaskan adanya kesenjangan yang kritis, yakni ekosistem API *multi-tenant* saat ini belum memiliki mekanisme *inline* yang mampu mengintegrasikan validasi kepemilikan objek dan evaluasi sinyal kontekstual secara dinamis dalam satu waktu.

Paradigma *Zero Trust Architecture* (ZTA) menjawab kelemahan tersebut melalui prinsip "*never trust, always verify*", yang mewajibkan verifikasi akses berkelanjutan berdasarkan evaluasi dinamis atas identitas pengguna, status perangkat, dan konteks lingkungan pada setiap *request* [6]. ZTA memiliki potensi sinergi yang sangat besar dengan *Context-Aware Authorization* (CAA), yang memperhitungkan atribut kontekstual secara *real-time* untuk keputusan otorisasi. Namun, irisan keduanya, khususnya pemanfaatan persepsi kontekstual untuk mitigasi BOLA, masih menjadi dimensi yang paling minim dieksplorasi dalam literatur [5] [7]. Untuk mengisi kesenjangan tersebut, penelitian ini mengembangkan mekanisme *Context-Aware Authorization* berbasis ZTA pada REST API *multi-tenant* dengan mengombinasikan *Hard Violation* (validasi kepemilikan *tenant* mutlak) dan *Soft Violation* (akumulasi sinyal anomali IP, waktu, dan *velocity*) ke dalam kalkulasi *Risk Score*. Kinerja sistem ini kemudian dievaluasi secara komparatif untuk mengukur tingkat efektivitas keamanan beserta *overhead latency* komputasinya, guna menyajikan bukti empiris yang terukur atas mitigasi BOLA secara *real-time*.

1.2 Rumusan Masalah

1. Bagaimana kerentanan mekanisme otorisasi statis konvensional dalam menahan skenario serangan *Broken Object Level Authorization* (BOLA) pada ekosistem REST API yang bersifat *multi-tenant*?

2. Bagaimana merancang mekanisme *Context-Aware Authorization* berbasis *Zero Trust* yang mengombinasikan deteksi *Hard Violation* (ketidaksesuaian identitas *tenant*) dan *Soft Violation* (anomali kontekstual) ke dalam perhitungan *Risk Score* untuk memitigasi serangan BOLA?
3. Bagaimana efektivitas dan dampak performa dari implementasi *Zero Trust Context-Aware Authorization* tersebut ketika dievaluasi secara langsung terhadap simulasi serangan BOLA?

1.3 Tujuan Proyek

1. Menganalisis kerentanan mekanisme otorisasi statis terhadap serangan *Broken Object Level Authorization* (BOLA) pada arsitektur REST API *multi-tenant*.
2. Merancang dan mengimplementasikan mekanisme *Context-Aware Authorization* berbasis *Zero Trust* yang mengkombinasikan deteksi *Hard Violation* dan *Soft Violation* melalui perhitungan *Risk Score* menggunakan *Open Policy Agent* (OPA).
3. Mengevaluasi efektivitas dan dampak performa dari implementasi *Context Aware Authorization* berbasis *Zero Trust* dalam meningkatkan keamanan API dari serangan BOLA yang diukur melalui metrik tingkat deteksi (*True Positive Rate* dan *False Positive Rate*), tingkat isolasi (*Cross-Tenant Blocking Rate*), serta penambahan beban waktu komputasi (*overhead latency*).

1.4 Manfaat Proyek

Secara akademis, penelitian ini memberikan kontribusi berupa bukti empiris terukur mengenai efektivitas mekanisme *Context-Aware Authorization* berbasis *Zero Trust* dalam memitigasi serangan BOLA pada REST API *multi-tenant*. Di samping itu, penelitian ini menghasilkan *Proof of Concept* arsitektur *Zero Trust* yang mengintegrasikan *Hard Violation* dan *Soft Violation* ke dalam kalkulasi *Risk Score*, beserta dataset hasil evaluasi yang dapat direplikasi dan dijadikan landasan bagi penelitian lanjutan.

Secara praktis, penelitian ini menyediakan referensi implementasi konkret bagi pengembang dan tim keamanan dalam merancang mekanisme otorisasi berbasis *Zero Trust* pada platform yang mengelola data sensitif lintas organisasi. Pola desain *Policy-as-Code* menggunakan *Open Policy Agent* (OPA) yang dihasilkan memungkinkan kebijakan keamanan diperbarui secara independen tanpa mengubah logika bisnis aplikasi, sehingga lebih mudah dipelihara dan diaudit.

1.5 Ruang Lingkup dan Batasan

Agar penelitian tetap fokus dan realistis, proyek ini dibatasi pada ruang lingkup berikut.

1. Pengujian keamanan secara eksklusif dibatasi pada mitigasi serangan BOLA, tidak mencakup jenis serangan API lain di luar BOLA, seperti *Broken Function Level Authorization* (BFLA), injeksi, atau *Denial of Service*.
2. Purwarupa yang dikembangkan adalah sistem REST API berarsitektur *multi-tenant* yang menggunakan pendekatan *shared database, shared schema*.
3. Arsitektur keamanan terintegrasi dikembangkan menggunakan Node.js (Express.js) sebagai *middleware* (PEP), Keycloak sebagai *Identity Provider* penghasil JWT, *Open Policy Agent* (OPA) untuk evaluasi logika kebijakan deklaratif Rego, Redis sebagai manajemen konteks *in-memory* (validasi *velocity*, profil IP historis, *blacklist* JWT), serta Grafana Loki untuk pemantauan log aktivitas.
4. Penentuan bobot *Risk Score* dan angka ambang batas (*threshold*) pada evaluasi *Soft Violation* ditetapkan secara statis (*rule-based*) melalui *Policy-as-Code*, tidak menggunakan algoritma adaptif berbasis historis atau *Machine Learning*.
5. Jumlah *tenant* dan volume data yang diuji berskala kecil, sehingga hasil evaluasi performa tidak dapat digeneralisasi langsung ke sistem berskala produksi.

BAB II TINJAUAN PUSTAKA

2.1 Landasan Teori

2.1.1. REST API dan Arsitektur Multi-Tenant

a. Karakteristik dan Permukaan Serangan REST API

Representational State Transfer (REST) merupakan gaya arsitektur perangkat lunak yang memanfaatkan protokol HTTP/HTTPS sebagai media komunikasi utama antar-sistem. Karakteristik paling krusial dari REST API adalah sifatnya yang stateless, yang menegaskan bahwa server tidak menyimpan konteks sesi dari client. Setiap request yang dikirimkan oleh client wajib membawa seluruh informasi dan kredensial yang diperlukan secara mandiri [2].

Dari perspektif keamanan, sifat stateless menciptakan permukaan serangan (attack surface) yang luas karena setiap endpoint API dapat diakses langsung melalui jaringan. Mekanisme pemeriksaan keamanan tidak cukup dilakukan sekali di awal, melainkan harus dievaluasi secara ketat pada setiap request yang masuk untuk mencegah manipulasi parameter[2].

b. Arsitektur Multi-Tenancy dan Model Shared Database

Arsitektur multi-tenant merupakan pendekatan perancangan di mana satu instansi aplikasi melayani banyak kelompok pengguna terpisah yang disebut penyewa (tenant). Terdapat tiga model isolasi yang umum diterapkan:

1. Shared Database, Shared Schema: satu basis data dan satu skema untuk semua tenant, dibedakan oleh kolom tenant_id. Paling efisien dari sisi biaya namun membutuhkan kontrol logis yang ketat.
2. Shared Database, Separate Schema: satu basis data dengan skema terpisah per tenant. Menawarkan isolasi lebih kuat namun menambah kompleksitas migrasi.
3. Separate Database: basis data dedikasi per tenant. Isolasi maksimal dengan biaya operasional tertinggi.

Proyek ini mengadopsi model shared database, shared schema dengan kolom kode_wilayah sebagai diskriminator tenant, mengingat efisiensinya untuk sistem statistik BPS multi-wilayah.

c. Isolasi Data dan Kerangka Hukum UU PDP

Isolasi data merupakan mekanisme kontrol teknis untuk menjamin batasan logis pada arsitektur multi-tenant. Kegagalan dalam membangun sistem isolasi yang kuat berpotensi memicu kebocoran data lintas-tenant (*cross-tenant data leakage*). Berdasarkan Undang-Undang No. 27 Tahun 2022 tentang Perlindungan Data Pribadi (UU PDP), data kependudukan mikro yang bersifat sensitif, seperti NIK, nama lengkap, status kemiskinan, dan kepemilikan aset wajib dilindungi oleh Pengendali Data Pribadi dengan enkripsi dan kontrol akses tingkat tinggi.

Konsekuensi hukum dari kebocoran data mencakup sanksi administratif, denda masif, hingga tuntutan pidana. Oleh karena itu, isolasi data bukan sekadar praktik rekayasa perangkat lunak yang baik, melainkan sebuah kewajiban kepatuhan regulasi (*compliance requirement*) yang mendasar.

2.1.2. Kerentanan Broken Object Level Authorization (BOLA)

a. Klasifikasi BOLA dalam OWASP API Security Top 10

Broken Object Level Authorization (BOLA) telah menduduki posisi teratas dalam OWASP API Security Top 10 sejak tahun 2019 dan dipertahankan pada edisi 2023 sebagai API1:2023. BOLA diklasifikasikan sebagai kegagalan fatal pada mekanisme otorisasi di sisi server saat menangani akses ke objek data spesifik[8].

Perbedaan mendasar BOLA dengan kerentanan lain adalah lokasi kegagalannya: BOLA terjadi pada tingkat objek, bukan tingkat endpoint. Artinya, proses autentikasi pengguna yang sudah benar sekalipun tidak mampu mencegahnya. BOLA juga berbeda dari Broken Function Level Authorization (BFLA) yang terjadi ketika penyerang mengakses fungsi atau endpoint yang seharusnya tidak dapat diakses sama sekali[8].

b. Mekanisme Eksploitasi BOLA pada REST API Multi-Tenant

Serangan BOLA terjadi karena pengembang mengasumsikan bahwa pengguna yang telah berhasil login dapat dipercaya untuk meminta data apapun. Penyerang memanfaatkan celah ini dengan mengeksploitasi endpoint API yang mengekspos pengenalan objek (*Object ID*) secara langsung pada URI atau bodi HTTP[2].

Santos Filho dkk. (2025) menunjukkan kompleksitas deteksi serangan BOLA pada REST API yang disebabkan oleh sifat logika bisnis yang sangat dinamis dan beragam. Penelitian tersebut memvalidasi kebutuhan mekanisme pertahanan yang bersifat *context-aware* dan tertanam dalam alur pemrosesan request (middleware).

Sebagai contoh dalam konteks sistem BPS multi-tenant, penyerang yang memiliki token valid Kabupaten A dapat mengirimkan request[9]:

GET /api/v1/kabupaten-b/penduduk/1002

Jika backend gagal memvalidasi relasi kepemilikan antara identitas token dengan objek tersebut, maka server akan mengembalikan data sensitif kepada penyerang[9]. membuktikan secara empiris bahwa aplikasi e-commerce dan e-banking di Indonesia masih rentan terhadap serangan BOLA akibat ketergantungan berlebihan pada keamanan sisi klien yang mudah ditembus[9].

2.1.3. Arsitektur Zero Trust (ZTA)

a. Prinsip Utama Berdasarkan NIST SP 800-207

Model keamanan tradisional mengadopsi pendekatan *perimeter defense*, di mana sistem memercayai entitas yang berada di dalam jaringan internal secara implisit. NIST SP 800-207 memperkenalkan Arsitektur Zero Trust (ZTA) yang beroperasi di bawah asumsi konstan bahwa ancaman sudah berada di dalam jaringan. ZTA menegakkan tiga prinsip utama[6].

1. Assume Breach: Selalu mengasumsikan jaringan telah dikompromikan; setiap request diperlakukan sebagai berpotensi berbahaya.
2. Verify Explicitly: Selalu lakukan autentikasi dan otorisasi secara eksplisit berdasarkan seluruh atribut yang tersedia (identitas, lokasi, perangkat, anomali perilaku).
3. Use Least Privilege: Batasi akses hanya pada sumber daya yang dibutuhkan pada saat itu (*Just-In-Time Access*).

b. Komponen Logis PEP dan PDP Berdasarkan NIST SP 800-207A

NIST SP 800-207A mempertegas pembagian komponen logis kontrol akses untuk implementasi ZTA pada lingkungan cloud-native [10]:

1. Policy Enforcement Point (PEP): Komponen gerbang (*gatekeeper*) yang mencegah setiap koneksi antara client dan resource. PEP mengekstrak parameter request, mengirimkannya ke PDP, dan mengeksekusi keputusan otorisasi secara mutlak.
2. Policy Decision Point (PDP): Otak dari arsitektur keamanan. PDP mengevaluasi kebijakan dengan mencocokkan identitas subjek dan atribut lingkungan, kemudian mengeluarkan keputusan *allow* atau *deny*.

Pemisahan PEP dan PDP ini merupakan fondasi dari desain Express.js middleware (PEP) dan Open Policy Agent (PDP) pada sistem yang dibangun dalam proyek ini.

2.1.4. Otorisasi Berbasis Konteks (Context-Aware Authorization)

a. Evolusi Kontrol Akses: RBAC, ABAC, dan CBAC

Model kontrol akses berbasis peran (Role-Based Access Control / RBAC) mengunci hak akses berdasarkan jabatan pengguna. Kelemahannya, jika kredensial dicuri, penyerang dapat mengeksploitasi akses sepenuhnya tanpa dicurigai. NIST SP 800-162[11] merumuskan Attribute-Based Access Control (ABAC) sebagai evolusi dari RBAC yang mengevaluasi variabel atribut secara dinamis, bukan hanya peran.

Model ini berkembang lebih lanjut menjadi *Context-Based Access Control* (CBAC) atau Otorisasi Berbasis Konteks, yang tidak hanya memeriksa identitas pengguna tetapi juga mengevaluasi variabel lingkungan dinamis saat request dikirimkan: alamat IP, waktu akses, frekuensi aktivitas, dan sidik jari perangkat digital.

b. Klasifikasi Pelanggaran: Hard Violation dan Soft Violation

Mengacu pada prinsip *Global Admissibility Predicate* dalam literatur Zero Trust adaptif, pelanggaran terhadap parameter keamanan dinamis dibagi menjadi dua kategori [6][10]:

1. Hard Violation (Pelanggaran Mutlak): Pelanggaran deterministik yang secara langsung mengindikasikan serangan, yaitu ketidaksesuaian identitas tenant (*Tenant Mismatch*). Satu kali terdeteksi langsung memicu keputusan BLOCK. Ini adalah satu-satunya sinyal yang mendeteksi serangan BOLA secara langsung dalam sistem yang dikembangkan.
2. Soft Violation (Pelanggaran Probabilistik): Anomali perilaku yang mencurigakan namun belum tentu merupakan serangan, seperti perubahan alamat IP, akses di luar jam kerja, atau frekuensi request yang tidak wajar. Pelanggaran ini mengurangi skor kepercayaan (*trust score*) secara dinamis tanpa langsung memblokir akses.

c. Mekanisme Dual Threshold Decision dan Bobot Risk Score

Sistem otorisasi adaptif mengimplementasikan dua ambang batas keputusan (*dual thresholds*) berdasarkan akumulasi *risk score*.

Wilayah	Kondisi Skor	Respons Sistem
<i>Allow (Hijau)</i>	<i>Score < 50</i>	<i>Akses diberikan; log dikirim ke SOC Dashboard.</i>
<i>Block (Merah)</i>	<i>Score >= 50 atau Hard Violation</i>	<i>Akses ditolak (403 Forbidden); token dimasukkan ke blacklist Redis; security alert dikirim ke SOC.</i>

Threshold ditetapkan pada nilai 50 berdasarkan prinsip *Global Admissibility Predicate*: satu Hard Violation (Tenant Mismatch = +50) langsung memenuhi threshold, sementara kombinasi seluruh Soft Violation secara bersamaan (IP +20, waktu +10, velocity +15 = maks +45) tetap tidak mencapai threshold. Desain ini memastikan operator yang bekerja dari kondisi tidak biasa tidak diblokir secara keliru (meminimalkan False Positive secara struktural).

2.1.5. Komponen Teknologi Implementasi

a. Protokol OAuth 2.0, OpenID Connect, dan JSON Web Token

Dalam arsitektur Zero Trust, manajemen identitas disentralisasikan pada Identity Provider (IdP) melalui protokol OAuth 2.0 (RFC 6749) dan OpenID Connect (OIDC). Ketika pengguna berhasil autentikasi, IdP menerbitkan JSON Web Token (JWT) berdasarkan standar RFC 7519.

JWT mengemas informasi dalam payload terenkripsi Base64 yang berisi klaim-klaim terstruktur antara lain sub (identitas pengguna), tenant_id, role, dan exp (waktu kedaluwarsa). JWT dilengkapi tanda tangan digital kriptografis (RS256) yang menjamin integritas token sehingga gateway dapat memastikan token diterbitkan oleh IdP tepercaya dan tidak dimodifikasi[12].

Ahmad dkk. (2023) membandingkan keamanan JWT dengan PASETO pada RESTful API dan menemukan bahwa JWT terbukti aman terhadap serangan OWASP termasuk BOLA, namun perlu dikombinasikan dengan validasi kepemilikan objek di sisi server untuk mitigasi yang komprehensif[12].

b. Paradigma Policy-as-Code dengan Open Policy Agent (OPA)

Menyatukan logika otorisasi di dalam kode aplikasi utama (*hardcoded rules*) dinilai tidak efisien, sulit dipelihara, dan rentan kesalahan. Arsitektur Zero Trust modern menerapkan dekopling kebijakan melalui pendekatan *Policy-as-Code*: aturan keamanan ditulis secara terpisah dari kode bisnis dan dikelola sebagai artefak yang dapat di-version control[6].

Open Policy Agent (OPA) berperan sebagai mesin kebijakan terpusat yang menggunakan bahasa deklaratif Rego. OPA menerima input konteks berformat JSON dari PEP, mengevaluasi aturan Rego dalam milidetik, dan

mengembalikan keputusan otorisasi beserta risk score. Pendekatan ini sesuai dengan model ABAC dari NIST SP 800-162 dan mendukung tiga keunggulan: (1) pemisahan kebijakan dari logika bisnis, (2) auditabilitas karena aturan dapat di-version control, dan (3) hot-reload konfigurasi tanpa mengubah kode aplikasi[6][10].

c. Manajemen State Kontekstual dengan In-Memory Data Store (Redis)

Arsitektur Zero Trust menuntut verifikasi berkelanjutan (*continuous verification*) yang membutuhkan pelacakan riwayat perilaku pengguna secara real-time. Melakukan query berulang ke basis data relasional tradisional pada setiap request akan mengakibatkan lonjakan latensi yang tidak dapat diterima[6].

Komponen *in-memory data store* (Redis) beroperasi di RAM dengan latensi sub-milidetik, menjadikannya krusial untuk tiga fungsi dalam sistem ini: [10]

1. Penyimpanan profil IP historis per-pengguna untuk komparasi real-time pada setiap request.
2. Implementasi algoritma *Sliding Window Counter* untuk mendeteksi anomali frekuensi request (Velocity Check).
3. Penyimpanan daftar hitam JWT (*token blacklist*) untuk pencabutan akses paksa terhadap token yang dikompromikan sebelum masa kedaluwarsa habis.

2.2 Penelitian/Proyek Terkait

Penelitian	Fokus Utama	Pendekatan	Zero Trust Architecture	BOLA	Multi tenant	Context Aware	REST API	Keterbatasan
[13]	Perancangan arsitektur API Security Gateway dan model kontrol akses data untuk sistem full stack multi tenant.	Zero Trust Architecture, OAuth 2.1 + JWT, Open Policy Agent (OPA), GraphQL	✓	~	✓	~	✓	Penelitian ini membahas mitigasi unauthorized data access secara umum, tidak spesifik memitigasi serangan Broken Object Level Authorization (BOLA)

[14]	Perancangan kerangka adaptif ZeTHAA untuk verifikasi berkelanjutan pada sistem autentikasi statis	Model probabilistik, atribut dinamis (entropi, Beta-posterior), evaluasi kontekstual	✓	✗	✗	✓	~	Tidak membahas skenario multi-tenant dan kerentanan BOLA, belum ada validasi kemampuan generalisasi model ZeTHAA pada lingkungan multi tenant skala besar
[2]	Memodelkan perilaku REST API untuk mendeteksi potensi risiko keamanan dan serangan Broken Object Level Authorization (BOLA) secara otomatis melalui analisis log aktivitas web server.	Transformasi OpenAPI (v3.1.0) menjadi Colored Petri Nets (CPNs)	✗	✓	✗	~	✓	Sistem belum berjalan secara real-time (inline) di layer middleware, tidak mendukung arsitektur multi-tenant, dan tidak mengintegrasikan sinyal konteks perilaku (IP, time, velocity) sehingga memicu false positive yang tinggi.
[15]	Perancangan arsitektur otorisasi adaptif untuk perlindungan data sensitif secara aman dan kontekstual pada lingkungan terdistribusi.	Zero Trust Controller (PDP + Risk Engine) dan skema kriptografi Ciphertext-Policy Attribute-Based Encryption (CP-ABE)	✓	✗	✗	✓	✗	Sistem tidak ramah untuk arsitektur REST API web/mobile karena kebijakan tidak dipisahkan dari logika data (hardcoded dalam ciphertext), serta belum mendukung isolasi data lintas multi-tenant.
[16]	Implementasi Dynamic RBAC pada lingkungan cloud multi-tenant untuk mengisolasi role pengguna secara real time berdasarkan atribut	Keycloak, OIDC/OAuth 2.0, JWT, dan RBAC	✗	✗	✓	~	~	Tidak mengintegrasikan prinsip Zero Trust, konteks otorisasi terbatas pada atribut departemen, pengujian pada skala kecil lingkungan lokal tanpa validasi REST API

Penelitian ini	Implementasi dan evaluasi Context-Aware Authorization berbasis Zero Trust untuk memitigasi serangan BOLA pada sistem multi-tenant.	OPA (Open Policy Agent), Keycloak, Redis, dan evaluasi berbasis risiko (Hard vs Soft Violation).	✓	✓	✓	✓	✓	Diuji pada skala Proof of Concept (PoC) simulasi mandiri terbatas (3 skenario pengujian) dengan penentuan threshold risiko yang bersifat statis-deklaratif
----------------	--	--	---	---	---	---	---	--

BAB III METODOLOGI

3.1 Tahapan Pelaksanaan

Penelitian ini mengembangkan Proof of Concept (PoC) sistem otorisasi berbasis Context-Aware Zero Trust pada REST API Multi-Tenant. Kontribusi utama penelitian mencakup: (1) implementasi mekanisme Context-Aware Authorization yang mengevaluasi atribut kontekstual pada setiap request, (2) implementasi model Risk-Adaptive Authorization yang mengklasifikasikan pelanggaran ke dalam Hard Violation dan Soft Violation, (3) implementasi komponen arsitektur Zero Trust sesuai NIST SP 800-207 [2] menggunakan teknologi standar industri, dan (4) evaluasi efektivitas mekanisme otorisasi terhadap skenario serangan Broken Object Level Authorization (BOLA) sebagai use case pengujian utama. Pelaksanaan dilakukan melalui lima tahapan yang disusun secara sistematis.

3.1.1. Studi Literatur dan Analisis Kebutuhan

Tahapan ini bertujuan untuk mengidentifikasi *state-of-the-art* dalam mitigasi BOLA dan memetakan *research gap* yang akan diisi oleh proyek ini.

Aktivitas yang dilakukan:

- a. Kajian standar dan kerangka kerja industri – Menelaah OWASP API Security Top 10 (2023) untuk memahami karakteristik ancaman BOLA sebagai risiko keamanan API nomor satu, serta NIST SP 800-207 sebagai acuan arsitektur *Zero Trust* yang mensyaratkan evaluasi konteks jaringan, identitas, dan perilaku pada setiap *request*
- b. Kajian studi kasus kebocoran data akibat BOLA Menganalisis temuan [9] yang membuktikan secara empiris bahwa aplikasi E-Commerce dan E-Banking di Indonesia masih rentan terhadap serangan BOLA. Penelitian tersebut menyoroti ketergantungan berlebihan pada keamanan *Client-Side* (Jailbreak Detection, SSL Pinning) yang mudah ditembus, sehingga memvalidasi urgensi perlindungan di sisi *Server/Middleware*
- c. Kajian arsitektur API Security Gateway Multi-Tenant Menganalisis *framework* keamanan API Gateway dari [13] yang mengintegrasikan *Zero Trust Architecture*, OAuth 2.1, dan *policy engine* pada sistem Multi-Tenant berskala jutaan *request*. Dari penelitian ini diadopsi konsep *Threshold-Based Decision Function* (Equation 1 dalam paper), di mana

- keputusan otorisasi diambil melalui akumulasi bobot atribut kontekstual (*weighted score*) yang dibandingkan dengan nilai ambang batas (*threshold*)
- d. Kajian model *Risk-Adaptive Authorization* – Mempelajari model otorisasi adaptif berbasis risiko yang menggunakan analisis berkelanjutan terhadap karakteristik perilaku pengguna, parameter perangkat, konteks jaringan, dan kriticalitas sumber daya untuk membentuk indikator risiko integral secara *real-time*. Pendekatan ini menjadi acuan utama desain algoritma *Risk Scoring* pada *engine* yang dikembangkan
 - e. Kajian deteksi BOLA pada REST API – Menelaah Santos [2] yang menunjukkan kompleksitas deteksi serangan BOLA pada REST API akibat sifat logika bisnis yang sangat dinamis. Temuan ini memvalidasi kebutuhan mekanisme pertahanan yang bersifat *context-aware* dan tertanam di dalam alur pemrosesan *request (middleware)*
 - f. Kajian model Zero Trust Adaptive Authentication (ZeTHAA) Menganalisis framework *Zero-Trust Hybrid Adaptive Authentication* (ZeTHAA) dari [14] yang diterbitkan di IEEE Access. Paper ini memperkenalkan konsep Global Admissibility Predicate sebuah predikat formal yang memisahkan secara tegas antara Hard Violation (pelanggaran mutlak yang tidak dapat dikompensasi oleh sinyal konteks lainnya) dan Probabilistic Soft Violation (deviasi perilaku yang hanya menurunkan kepercayaan namun tidak langsung memblokir akses)
 - g. Analisis kebutuhan fungsional dan non-fungsional: berdasarkan hasil kajian di atas, diturunkan kebutuhan sistem berikut:

Kebutuhan Fungsional:

1. REST API multi-tenant dengan minimal 3 *endpoint* CRUD.
2. Autentikasi terpusat menggunakan *Identity Provider* (Keycloak) yang menerbitkan JSON Web Token (JWT) dengan *claim*: *user_id*, *tenant_id*, dan *role*
3. *Policy Engine* berbasis Open Policy Agent (OPA) yang mengevaluasi kebijakan otorisasi menggunakan bahasa deklaratif Rego, memisahkan logika kebijakan dari logika bisnis (*Policy-as-Code*)

4. *Context-Aware Authorization* melalui *Expres.js middleware* yang bertindak sebagai *Policy Enforcement Point (PEP)*), mengumpulkan konteks *request* dan mengeksekusi keputusan dari *Policy Engine*
5. Validasi kepemilikan *tenant (Tenant Ownership Validation)* sebagai kontrol utama mitigasi BOLA, diimplementasikan sebagai *Hard Violation* sesuai prinsip *Gloval Admissibility Predicate*
6. Evaluasi sinyal kontekstual (*IP Address Profiling, Access Time Analysis, Velocity Check*) sebagai mekanisme *Continuous Verification* sesuai prinsip *Zero Trust*
7. Mekanisme *JWT Blacklisting* menggunakan Redis untuk pencabutan akses paksa (*forced revocation*) terhadap token yang terdeteksi melakukan pelanggaran
8. *Dashboard SOC (Security Operations Center)* yang menampilkan metrik keamanan secara *real-time* melalui koneksi *WebSocket (Socket.IO)*
9. *Toggle switch* untuk mengaktifkan/menonaktifkan mekanisme otorisasi dalam evaluasi komparatif (*baseline vs protected*)

Kebutuhan Non-Fungsional

1. *Overhead latency middleware* keseluruhan pipeline otorisasi ≤ 100 ms per request
2. *True Positive Rate* deteksi serangan BOLA $\geq 95\%$
3. *False Positive Rate* $\leq 5\%$
4. *Cross-Tenant Blocking Rate* $\geq 99\%$ (mengacu *benchmark* Mao dkk yang mencapai 99,1%)

3.1.2. Perancangan Sistem dan Arsitektur Keamanan

Hasil analisis kebutuhan dari Tahap 1 diterjemahkan ke dalam rancangan arsitektur teknis. Aktivitas yang dilakukan:

a. Perancangan skema database Multi-Tenant

Menggunakan pendekatan *shared database, shared schema* dengan kolom *tenant_id* (berupa *kode_wilayah*) sebagai diskriminator. Simulasi mengangkat skenario Sistem Informasi Data Sensus Kependudukan

Terpadu yang digunakan oleh beberapa satuan kerja BPS tingkat Kabupaten/Kota. Setiap *record* data penduduk (NIK, nama, status kemiskinan, aset) terikat secara ketat pada satu *tenant* (wilayah kerja) tertentu. Pemilihan doain ini didasarkan pada urgensi nyata (kebocoran data kependudukan *by name by address* lintas-wilayah merupakan pelanggaran UU Perlindungan Data Pribadi sesuai UU No.27 Tahun 2022) dan berpotensi mengancam keamanan nasional

b. Perancangan Identity Provider (Keycloak)

Merancang Keycloak sebagai *Identity Provided* (IdP) terpusat yang bertanggung jawab atas autentikasi pengguna dan penerbitan JWT (*Access Token*). Keycloak dipilih karena: (a) merupakan implementasi standar OpenID Connect (OIDC) dan OAuth 2.0 yang sesuai dengan rekomendasi NIST SP 800-207 untuk komponen *Identity Provider*; (b) mendukung konfigurasi *realm* dan *client* yang memungkinkan penyisipan *custom claim* (*tenant_id*, *role*) ke dalam token JWT, dan (c) bersifat *open-source* sehingga mendukung transparansi dan reproduibilitas akademis. Token JWT yang diterbitkan oleh Keycloak memuat tiga *claim* utama: *user_id* (identitas pengguna), *tenant_id* (identitas tenant/wilayah kerja), dan *role* (peran otorisasi)

c. Perancangan Policy Engine (Open Policy Engine / OPA)

Merancang *Policy Engine* (PE) yang mengevaluasi kebijakan otorisasi menggunakan bahasa deklaratif Rego. OPA dipilih berdasarkan justifikasi berikut:

1. Pemisahan kebijakan dari logika bisnis OPA mengimplementasikan prinsip *Policy-as-Code* dimana aturan otorisasi ditulis secara terpisah dari kode aplikasi, sesuai dengan model ABAC yang didefinisikan dalam NIST SP 800-162
2. Kesesuaian dengan arsitektur Zero Trust modern
3. Auditabilitas dan reproducibility Kebijakan Rego bersifat deklaratif, dapat di-*version control*, dan mudah diaudit, mendukung transparansi yang diperlukan dalam penelitian akademis

d. Perancangan Policy Enforcement Point (Express.js Middleware)

Merancang Express.js *middleware* kustom yang bertindak sebagai *Policy Enforcement Point* (PEP) sesuai model NIST SP 800-207. *Middleware* ini

bertugas: (a) mencegah setiap request yang masuk, (b) mengekstrak konteks request (JWT claims, alamat IP, timestamp, frekuensi akses), (c) melakukan query ke OPA untuk mendapatkan keputusan otorisasi, dan (d) menegakkan (enforce) keputusan tersebut.

e. Perancangan model Risk-Adaptive Authorization

Merancang model otorisasi adaptif yang mengadopsi konsep Threshold-Based Decision Function dari Mao dkk. (2025), model Risk-Adaptive Authorization dari Kostiuk dkk. (2026), serta konsep Global Admissibility Predicate dari Krishnan & Sreeja (2026). Model ini memisahkan faktor risiko menjadi dua kategori yang berbeda secara fundamental :

1. Hard Violation : pelanggaran yang bersifat mutlak dan tidak dapat dikompensasi oleh faktor konteks lainnya. Satu kali terdeteksi langsung memenuhi *threshold* dan memblokir akses. Diimplementasikan pada Tenant Ownership (kesesuaian *tenant_id* JWT dengan *tenant_id* objek target)
2. Soft Violation
 - i. Dimensi Jaringan: *IP Address Profiling* – kesesuaian *IP request* dengan profil historis pengguna yang tersimpan di Redis
 - ii. Dimensi Perilaku (Waktu): *Time Window Analysis* – kesesuaian waktu akses dengan jam operasional normal;
 - iii. Dimensi Perilaku (Frekuensi): *Velocity Check* – deteksi frekuensi *request* abnormal menggunakan *sliding window counter* pada Redis

f. Perancangan Context Store (Redis) Merancang Redis sebagai Context Information Repository yang menyimpan: (a) profil IP historis per-pengguna, (b) sliding window counter untuk kalkulasi Velocity, dan (c) daftar hitam JWT (blacklist) untuk pencabutan akses paksa. Redis dipilih karena karakteristik in-memory yang mendukung latensi rendah pada operasi baca/tulis yang berfrekuensi tinggi.

g. Perancangan dashboard SOC interaktif Mendesain antarmuka dashboard yang terdiri dari Client/Attacker View (simulasi request) dan SOC Security Dashboard (visualisasi real-time). Dashboard terhubung ke backend melalui

Socket.IO untuk menerima event keamanan secara real-time. Dilengkapi toggle switch untuk mengaktifkan/menonaktifkan pipeline otorisasi saat demonstrasi evaluasi komparatif

3.1.3. Implementasi dan Pengembangan

Desain arsitektur direalisasikan ke dalam kode program. Rincian komponen implementasi:

Komponen	Peran NIST SP 800-207	Teknologi	Deskripsi
Identity Provider	Identity Provider (IdP)	Keycloak	Autentikasi pengguna via OpenID connect. Menerbitkan JWT dengan <i>claim</i> : user_id, tenant-id, role
API Gateway / Middleware	Policy Enforcement Point (PEP)	Exress.js Middleware	Mencegat <i>request</i> , mengekstrak konteks, melakukan <i>query</i> ke OPA, dan menegakkan keputusan otorisasi
Policy Engine	Policy Engine (PE)	Open Policy Agent (OPA)	Mengevaluasi kebijakan otorisasi yang ditulis dalam bahasa Rego. Menerima input konteks dari PEP dan mengembalikan keputusan <i>allow/deny</i> beserta <i>risk score</i> .
Context Store	Context Information Repository	Redis via ioredis	Menyimpan profil IP historis per-tenant, <i>sliding window counter</i>

			untuk kalkulasi <i>Velocity</i> , dan daftar hitam JWT (<i>blacklist</i>)
Database	Protected Resource	SQLite via better-sqlite	Data <i>seed</i> untuk minimal 3 tenant (wilayah BPS Kabupaten/Kota),, masing-masing memiliki data mikro penduduk sensitif (NIK, nama, status kemiskinan, aset) yang terisolasi
Frontend Dashboard	-	React.js (Vite)	Tampilan <i>Split-Screen</i> : sisi kiri (<i>Client View</i>), sisi kanan (<i>SOC Dashboard</i> dengan <i>Risk Score Meter</i> dan <i>log</i>)
Komunikasi Real-Time	-	Socket.IO	Setiap event keamanan (ALLOW/BLOCK) dikirim secara real-time dari PEP ke Dashboard SOC
Toggle Zero Trust	-	React State + Backend Flag	Tombol toggle untuk mengaktifkan/menonaktifkan pipeline otorisasi, memungkinkan evaluasi komparatif baseline vs protected

3.1.4. Pengujian dan Evaluasi

Pengujian dilakukan menggunakan pendekatan Dual-Scenario Testing. Detail skenario dan metrik diuraikan pada Subbab 3.3

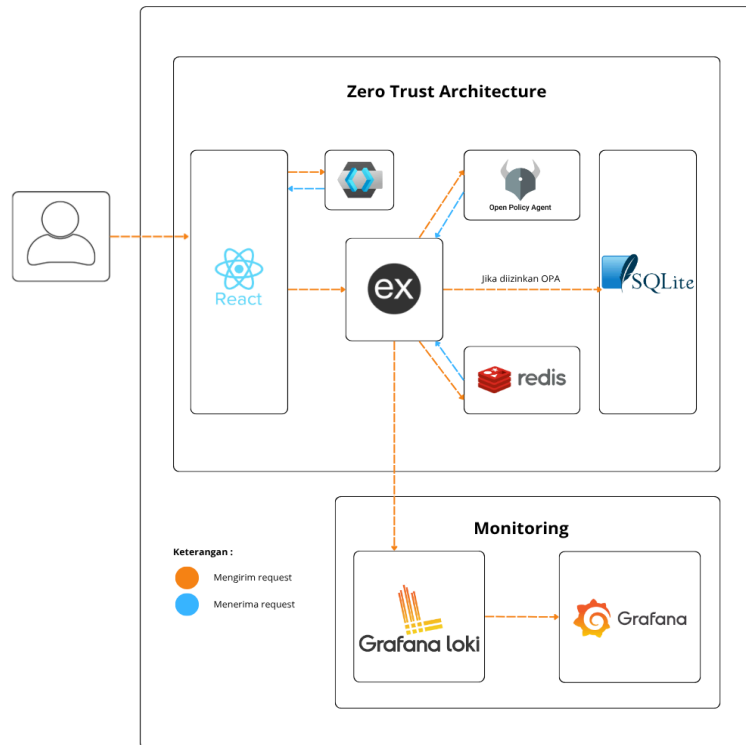
3.1.5. Dokumentasi dan Penyusunan Laporan

- a. Dokumentasi teknis – intruksi instalasi, konfigurasi, dan pengoperasian PoC (README)
- b. Analisis hasil – tabulasi kuantitatif dari metrik evaluasi (TPR, FPR, *Latency Overhead*) beserta pembahasan
- c. Kesimpulan dan saran – rangkuman temuan, keterbatasan PoC, dan rekomendasi pengembangan (misalnya: integrasi *Machine Learning* untuk deteksi anomali adaptif, atau dynamic threshold berbasis data historis)
- d. Lampiran – *Source code* kunci (*middleware*, *policy JSON*, konfigurasi *Redis*), dan *screenshot* hasil pengujian, dan demonstrasi Dashboard SOC

3.2 Arsitektur/Desain Sistem

3.2.1. Diagram Arsitektur Sistem

Arsitektur dirancang berdasarkan model komponen *Zero Trust* dari NIST SP 800-207, di mana *Context-Aware Authorization Engine* bertindak sebagai Policy Enforcement Point (PEP) yang mencegah setiap *request* sebelum mencapai logika bisnis inti. Sistem menggunakan Redis sebagai *in-memory cache* untuk mendukung kalkulasi *Velocity* dan *JWT Blacklisting* secara efisien



Gambar 3.1 Diagram Arsitektur Sistem

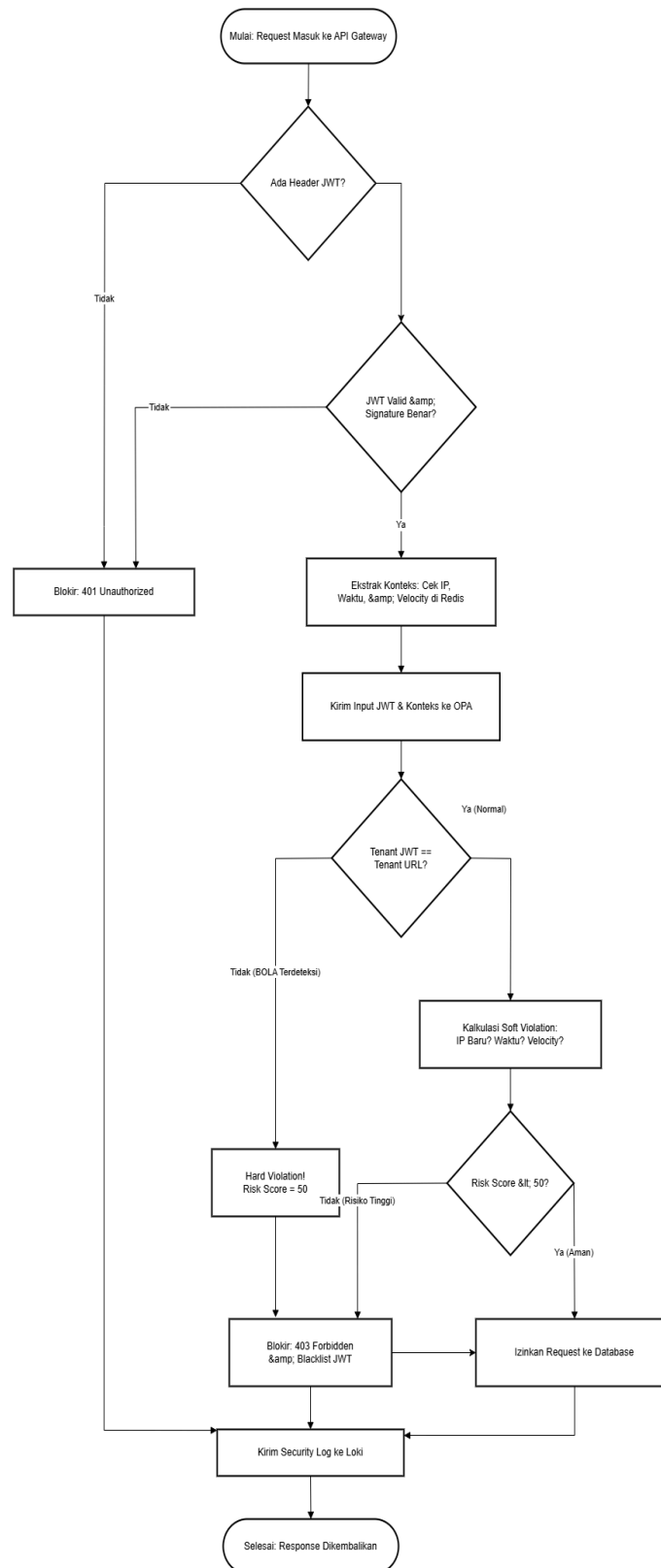
Keterangan:

- Keycloak berperan sebagai *Identity Provide* yang mengautentikasi pengguna dan menerbitkan JWT. Keycloak tidak terlibat dalam keputusan otorisasi *per-request*, tugasnya selesai setelah token diterbitkan
- Express.js Middleware (PEP) mencegat setiap *request*, mengekstrak konteks (JWT *claims*, IP *address*, *timestamp*, frekuensi akses dari Redis), dan mengirimkan konteks tersebut ke OPA untuk dievaluasi
- OPA (Policy Engine) mengevaluasi kebijakan Rego berdasarkan konteks yang diterima dari PEP. Keputusan otorisasi (*allow/deny*) beserta *risk score* dikembalikan ke PEP untuk ditegakkan
- Redis (Context Store) menyimpan data kontekstual yang diperlukan untuk evaluasi: profil IP historis, *velocity counter (sliding window)*, daftar hitam JWT
- Toggle Switch memungkinkan *pipeline* otorisasi diaktifkan/dinonaktifkan untuk evaluasi komparatif *baseline* vs *protected*

3.2.2. Diagram Alir (Logika Context-Aware Authorization Engine)

Flowchart berikut menggambarkan detail alur pengambilan keputusan otorisasi pada setiap *request*, termasuk interaksi antara PEP, OPA, dan Redis. Keterangan alurnya sebagai berikut:

- a. Langkah 1-2 merupakan validasi autentikasi dasar berupa verifikasi kriptografis JWT dan pengecekan *blacklist* di Redis
- b. Langkah 3 mengumpulkan konteks *request* dari berbagai sumber, antara lain IP dari *header*, *timestamp* dari *server clock*, dan *velocity counter* dari Redis
- c. Langkah 4 mengirimkan seluruh konteks ke OPA sebagai input untuk evaluasi kebijakan Rego
- d. Langkah 5 merupakan kontrol mitigasi BOLA, jika *tenant_id* pada JWT tidak sesuai dengan *tenant_id* pada *resource*, *request* langsung ditolak (*Hard Violation*). Ini adalah satu-satunya langkah yang secara langsung mendeteksi serangan BOLA
- e. Langkah 6-7 merupakan evaluasi sinyal risiko kontekstual untuk *Continuous Verification* sesuai prinsip Zero Trust. Ketiga sinyal ini bukan indikator BOLA, melainkan indikator anomali perilaku yang memperkaya penilaian risiko secara holistik.



Gambar 3.2 Diagram Alir Pengambilan Keputusan Otorisasi

3.2.3. Justifikasi Bobot *Risk Score* dan *Threshold*

Desain bobot *Risk Score* dan nilai *threshold* didasarkan pada dua prinsip, yaitu klasifikasi tingkat keparahan pelanggaran (*Hard Violation* vs *Soft Violation*) dan referensi akademis yang tervalidasi

Variabel Konteks	Bobot	Klasifikasi	Peran dalam Sistem	Justifikasi Akademis
Tenant ID Mismatch	50	Hard Violation	Kontrol Mitigasi BOLA	Indikasi langsung serangan BOLA. menggunakan mekanisme isolasi berbasis Tenant ID (TID) sebagai kontrol utama (<i>cross-tenant blocking rate</i> 99,1%). Satu pelanggaran ini cukup untuk memblokir <i>request</i> [13]
Unknown IP Address	20	Soft Violation	Sinyal Kontekstual (<i>Continuous Verification</i>)	NIST SP 800-207 mensyaratkan evaluasi konteks jaringan. memasukkan “ <i>Network Context</i> ” sebagai salah satu pilar pembentuk indikator risiko. Sendiri tidak cukup untuk blokir, tetapi meningkatkan kecurigaan
Unusual Access Time	10	Soft Violation	Sinyal Kontekstual (<i>Continuous Verification</i>)	menggunakan “ <i>Behavioral Characteristics</i> ” (termasuk waktu akses) sebagai parameter perilaku. Bobot kecil karena akses di luar jam kerja bisa dilakukan oleh pengguna sah (lembur/perjalanan dinas)

Velocity Exceeded	15	Soft Violation	Sinyal Kontekstual (<i>Continuous Verification</i>)	mengklasifikasikan “ <i>High-frequency access manipulation</i> ” dan “ <i>Silent Probe Behavior</i> ” sebagai serangan utama yang wajib dideteksi [13]. Pola <i>automated enumeration</i> (BurpSuite Intruder) memiliki ciri frekuensi tinggi
-------------------	----	----------------	---	---

Nilai *threshold* 50 dirancang mengikuti prinsip Global Admissibility Predicate yang menyatakan bahwa keputusan blokir harus memiliki dua jalur yang terdistinsi secara formal:

- a. Jalur Hard Violation (Tenant Mismatch = +50): Satu kali pelanggaran kepemilikan objek langsung memenuhi threshold → **zero tolerance** terhadap serangan BOLA. Ini merupakan *non-compensable failure* per definisi
- b. Jalur Akumulasi Soft Violation (IP +20 + Waktu +10 + Velocity +15 = maks. +45): Kombinasi seluruh *Soft Violation* secara bersamaan tetap tidak mencapai threshold, sehingga operator yang bekerja dari IP baru saat lembur di lapangan tidak akan diblokir *False Positive* diminimalkan secara struktural, bukan hanya heuristik

Pada skala penelitian besar, *threshold* ditentukan secara dinamis menggunakan Machine Learning. Pada skala PoC ini, nilai 50 adalah titik optimal yang mengimplementasikan prinsip *Global Admissibility Predicate* secara ringan (*lightweight*) melalui *Policy-as-Code* JSON tanpa ketergantungan infrastruktur *Machine Learning*

3.2.4. Struktur Kebijakan Otorisasi (OPA Rego)

Kebijakan otorisasi diimplementasikan menggunakan bahasa deklaratif **Rego** pada OPA. Pendekatan ini sesuai dengan prinsip *Policy-as-Code* yang direkomendasikan oleh Mao dkk. dan kerangan ABAC dari NIST SP 800-162.

Berikut ini adalah rancangan struktur kebijakan Rego yang relevan :

```

package zerotrust.authorization

import rego.v1

default decision := {"allow": false, "risk_score": 0, "reason": "default_deny"}

# --- HARD VIOLATION: Kontrol Mitigasi BOLA ---
# Tenant Ownership Validation (non-compensable)
hard_violation if {
    input.jwt.tenant_id != input.resource.tenant_id
}

# --- SOFT VIOLATIONS: Sinyal Risiko Kontekstual ---
ip_score := 20 if { not input.context.ip_known } else := 0
time_score := 10 if { not input.context.within_work_hours } else := 0
velocity_score := 15 if { input.context.request_rate > data.policy.max_rpm } else := 0

total_risk := ip_score + time_score + velocity_score

# --- KEPUTUSAN OTORISASI ---
decision := {"allow": false, "risk_score": 50, "reason": "hard_violation_tenant_mismatch"} if {
    hard_violation
}

decision := {"allow": false, "risk_score": total_risk, "reason": "soft_violation_threshold"} if {
    not hard_violation
    total_risk >= data.policy.threshold
}

```

```
decision := {"allow": true, "risk_score": total_risk, "reason": "access_granted"} if {
    not hard_violation
    total_risk < data.policy.threshold
}
```

Keterangan:

- Kebijakan Rego memisahkan secara eksplisit antara `hard_violation` (kontrol BOLA) dan evaluasi `total_risk` (sinyal kontekstual)
- Konfigurasi bobot dan *threshold* disimpan dalam `data.policy` yang dapat diperbarui tanpa mengubah logika kebijakan (*hot-reload*)
- Keputusan dikembalikan sebagai objek JSON yang memuat `allow` (boolean), `risk_score` (numerik), dan `reason` (string) untuk keperluan logging dan visualisasi pada *SOC Dashboard*

3.3 Skenario Pengujian

3.3.1. Lingkungan Pengujian

Pengujian dilakukan pada aplikasi REST API simulasi yang dibangun secara mandiri menggunakan framework Node.js/Express dan React.js yang merepresentasikan Sistem Informasi Data Sensus Kependudukan Terpadu dengan arsitektur multi-tenant. Setiap tenant merepresentasikan satu satuan kerja BPS tingkat Kabupaten/Kota dengan data mikro penduduk sensitif (NIK, nama, status kemiskinan, aset) yang terisolasi secara ketat per wilayah kerja.

Aplikasi simulasi dirancang dengan tiga kondisi yaitu :

Kondisi	Deskripsi
No Authorization	Tanpa mekanisme otorisasi apapun digunakan pada Skenario A
Static Authorization	Otorisasi statis konvensional berbasis pengecekan kepemilikan sederhana tanpa konteks digunakan pada Skenario B

ZTA Context Aware otorisasi penuh berbasis Zero Trust dengan Hard Violation, Soft Violation, dan Risk Score digunakan pada Skenario C

Lingkungan pengujian terdiri dari komponen berikut:

- a. API Server sebagai target pengujian
- b. Open Policy Agent (OPA) sebagai policy engine ZTA
- c. Keycloak sebagai identity provider
- d. Redis dan SQLite untuk database
- e. Grafana Loki untuk pencatat log aktivitas

3.3.2. Skenario Pengujian

- a. Skenario A - Pengujian kondisi tanpa keamanan sama sekali

Tujuan: Membuktikan secara empiris bahwa sistem tanpa mekanisme otorisasi sepenuhnya rentan terhadap serangan BOLA lintas-tenant, dan mendokumentasikan Attack Success Rate sebagai nilai baseline perbandingan.

Aksi	Yang Terjadi
Attacker login sebagai User A, mendapatkan token valid	Login berhasil
Attacker akses resource milik sendiri <i>/api/orders/1001</i>	Berhasil, respon 200 OK
Attacker ganti ID secara manual <i>/api/orders/1002</i>	Data user lain bocor (200 OK)
Attacker enumerasi ID lain	Semua data berhasil diakses

- b. Skenario B - Pengujian kondisi dengan keamanan namun bukan Zero Trust Architecture

Tujuan: Mengevaluasi batas kemampuan mekanisme otorisasi statis konvensional dalam menahan serangan BOLA, sekaligus mengidentifikasi celah yang tidak dapat ditangani oleh pendekatan non-kontekstual

Aksi	Yang Terjadi
Attacker login sebagai User A, mendapatkan token valid	Login berhasil
Attacker akses resource milik sendiri <i>/api/orders/1001</i>	Berhasil, respon 200 OK
Attacker ganti ID secara manual <i>/api/orders/1002</i>	Ditolak, respon 403 Forbidden
Attacker enumerasi ID lain	Semua request ditolak

c. Skenario C - Pengujian dengan Context-Aware Authorization Berbasis ZTA

Tujuan: Mengevaluasi efektivitas penuh pipeline Context-Aware Authorization berbasis Zero Trust dalam memblokir serangan BOLA (Hard Violation) sekaligus mendeteksi anomali perilaku kontekstual (Soft Violation), serta mengukur dampak overhead latency terhadap performa sistem.

Sub-skenario C1 : Hard validation / Mitigasi BOLA langsung

Aksi	Yang Terjadi
Attacker login sebagai User A, mendapatkan token valid	Login berhasil
Attacker akses resource milik sendiri <i>/api/orders/1001</i>	Berhasil, respon 20V. C. Hu et al., "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," NIST, NIST SP 800-162, Jan. 2014. doi: 10.6028/NIST.SP.800-1620 OK, risk_score = 0

Attacker ganti ID secara manual <i>/api/orders/1002</i>	Ditolak, respon 403 Forbidden, risk_score = 50
Attacker enumerasi ID lain	Semua request ditolak
Sistem mencatat aktivitas mencurigakan di log	Alert tercatat di Grafana Loki dna bisa dimonitoring di Grafana

Sub-skenario C2 : Soft Validation / deteksi anomali

Aksi	Soft validation	Risk score	Yang Terjadi
Request dari IP yang tidak dikenal	IP Unknown	20	Allow
<i>Request di luar jam operasional</i>	Unusual Time	10	Allow
Request dengan frekuensi melebihi batas RPM (velocity exceeded)	Velocity	15	Allow
Kombinasi IP baru + velocity tinggi	IP Unknown + Velocity	35	Allow
Kombinasi seluruh Soft Violation sekaligus	IP Unknown + Unusual Time + Velocity	45	Allow
Hard Violation + seluruh Soft Violation	Semua kesalahan	100	Block

3.3.3. Parameter Keberhasilan

Untuk melihat keberhasilan dapat dilihat dari paramater :

- Akses unauthorized berhasil diblokir 100% → request ke object milik user lain ditolak

- b. Akses authorized tetap berjalan normal → User legitimate tidak terganggu (0% false block)
- c. Respon sistem terhadap serangan → Sistem mengembalikan HTTP 403 pada setiap pelanggaran
- d. Tidak semua *Soft violation* di anggap serangan
- e. Aktivitas serangan tercatat di log → 100% percobaan serangan masuk ke log monitoring

3.3.4. Evaluasi

Evaluasi dilakukan secara kuantitatif dengan membandingkan hasil ketiga skenario menggunakan empat metrik utama :

- a. True Positive Rate (TPR)

Mengukur proporsi serangan BOLA yang berhasil dideteksi dan diblokir oleh sistem

$$TPR = \frac{TP}{TP + FN} \times 100\%$$

- b. False Positive Rate (FPR)

Mengukur proporsi request legitimate yang secara keliru diblokir oleh sistem.

$$FPR = \frac{FP}{FP + TN} \times 100\%$$

- c. Cross-Tenant Blocking Rate (CTBR)

Mengukur efektivitas isolasi data antar-tenant, khusus untuk konteks multi-tenant pada sistem ini

$$CTBR = \frac{\text{Request lintas tenant yang diblokir}}{\text{Total request lintas tenant}} \times 100\%$$

- d. Overhead Latency

Mengukur penambahan waktu respons yang diakibatkan oleh keseluruhan pipeline otorisasi ZTA dibandingkan kondisi baseline tanpa otorisasi.

$$\text{Overhead} = t_{ZTA} - t_{baseline}$$

BAB IV JADWAL PELAKSANAAN

Berikut adalah rencana jadwal pelaksanaan proyek:

No	Kegiatan	Minggu 8-9	Minggu 10-11	Minggu 12-13	Minggu 14-16
1	Studi literatur & analisis kebutuhan				
2	Perancangan sistem/arsitektur keamanan				
3	Implementasi & pengembangan				
4	Pengujian & evaluasi				
5	Penyusunan laporan & presentasi				

BAB V PEMBAGIAN TUGAS

No	Nama Anggota	Peran	Tugas Utama
1	Anggita Cristin Meylani	Penyusun Pendahuluan (Bab I)	Menyusun latar belakang masalah terkait kerentanan BOLA pada arsitektur multi-tenant, merumuskan masalah, tujuan, manfaat, serta menentukan ruang lingkup dan batasan proyek.
2	Kaylla Zahrani	Analisis Penelitian Terkait (Bab II)	Mencari, menganalisis, dan menyusun studi komparasi dari penelitian atau proyek terdahulu untuk menemukan kesenjangan (gap) penelitian.
3	Muhammad Imaddudin Zaki	Perancang Skenario Pengujian (Bab III)	Merancang lingkungan pengujian perangkat keras dan lunak, menyusun skenario test case untuk simulasi serangan BOLA (Hard/Soft Violation), serta menentukan parameter keberhasilan sistem.
4	Nadia Nur Nisrina	Perancang Arsitektur & Pelaksanaan (Bab III)	Menyusun metodologi pelaksanaan proyek, merancang diagram alir (<i>engine logic</i>) dan arsitektur sistem, serta

			membuat formulasi bobot <i>risk score</i> dan kebijakan otorisasi menggunakan OPA Rego.
5	Nyimas Virna Salsa Lestari Risqia	Penyusun Landasan Teori (Bab II)	Menyusun landasn teori secara mendalam mengenai <i>REST API Multi-Tenant</i> , karakteristik kerentanan BOLA (OWASP API Security), konsep <i>Zero Trust Architecture</i> , serta <i>Context-Aware Authorization</i> .

DAFTAR PUSTAKA

- [1] OWASP Top 10 Team, “A01:2025 – Broken Access Control,” OWASP Top 10:2025. [Online]. Available: https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/
- [2] A. S. Filho, R. J. Rodríguez, and E. L. Feitosa, “Automated broken object-level authorization attack detection in REST APIs through OpenAPI to colored petri nets transformation,” *Int. J. Inf. Secur.*, vol. 24, no. 2, pp. 1–19, 2025, doi: 10.1007/s10207-024-00970-5.
- [3] O. A. S. P. Team, “API1:2023 Broken Object Level Authorization,” OWASP API Security Top 10. Accessed: Jun. 05, 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>
- [4] J. Lu and L. Li, “Detecting Broken Object-Level Authorization Vulnerabilities in Database-Backed Applications,” 2024, doi: 10.1145/3658644.3690227.
- [5] S. Xiao, Y. Ye, N. Kanwal, T. Newe, and B. Lee, “SoK : Context and Risk Aware Access Control for Zero Trust Systems,” vol. 2022, 2022, doi: 10.1155/2022/7026779.
- [6] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture NIST Special Publication 800-207”.
- [7] M. L. Gambo and A. Almulhem, “Zero Trust Architecture : A Systematic Literature Review,” 2025.
- [8] Open Web Application Security Project (OWASP), “OWASP API Security Top 10 2023,” OWASP API Security Project. [Online]. Available: <https://owasp.org/API-Security/>
- [9] Y. Muliono, “ScienceDirect Peeking Peeking and and Testing Testing Broken Broken Object Object Level Level Authorization Authorization Vulnerability onto and Mobile Vulnerability onto E-Commerce and E-Banking Mobile Applications Applications,” *Procedia Comput. Sci.*, vol. 179, no. 2020, pp. 962–965, 2021, doi: 10.1016/j.procs.2021.01.101.
- [10] Z. Butcher and Z. Butcher, “NIST Special Publication NIST SP 800-207A A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in

- Multi-Location Environments A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments”.
- [11] V. C. Hu, K. Scarfone, R. Kuhn, and K. Sandlin, “Guide to Attribute Based Access Control (ABAC) Definition and Considerations NIST Special Publication 800-162 Guide to Attribute Based Access Control (ABAC) Definition and Considerations”.
 - [12] P. Siber, P. Siber, P. Siber, and P. Siber, “Performance and Security Comparison of Json Web Tokens (JWT) and Platform Agnostic Security Tokens (PASETO) on RESTful APIs,” *2023 IEEE Int. Conf. Cryptogr. Informatics, Cybersecurity*, pp. 15–22, 2023, doi: 10.1109/ICoCICs58778.2023.10277377.
 - [13] M. Engineering, Y. Mao, and J. Li, “Research on API Security Gateway and Data Access Control Model for Multi-tenant Full-stack Systems,” pp. 43–47, 2025.
 - [14] V. Krishnan and C. S. Sreeja, “Provably Adaptive Trust Dynamics in Context-Aware Zero-Trust Systems : A Formal Framework for Continuous Verification,” *IEEE Access*, vol. 14, no. April, pp. 77839–77878, 2026, doi: 10.1109/ACCESS.2026.3695458.
 - [15] K. Routray and P. Bera, “ZTAAC : Zero Trust Adaptive Authorization with CP-ABE for Context-Aware Data Protection,” *2025 17th Int. Conf. Commun. Syst. NETworks*, pp. 814–816, 2025, doi: 10.1109/COMSNETS63942.2025.10885763.
 - [16] M. Kiran and N. Kumaresh, “Implementation of Dynamic Role Based Access Control in Multi-Tenant Cloud,” *2025 2nd Int. Conf. New Front. Commun. Autom. Manag. Secur.*, pp. 1–6, 2025, doi: 10.1109/ICCAMS65118.2025.11234495.